

Министерство Образования Республики Беларусь
УО Брестский Государственный Технический Университет
Кафедра интеллектуальных информационных технологий

Лабораторная работа № 5

По дисциплине: «Современные платформы программирования»

За 6 семестр

Выполнил:

Студент 3 курса

Группы ПО-12

Матюх И.М.

Проверил:

Кулик А.Д.

Цель: приобрести практические навыки разработки API и баз данных.

Вариант-12

Общее задание

1. Реализовать базу данных из не менее 5 таблиц на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;
 2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);
 3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;
 4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);
 5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт;
- Базу данные можно реализовать в любой СУБД (MySQL, PostgreSQL, SQLite и др.)
- 12) База данных Европейские футбольные чемпионаты

Код программы

Models.py

```
from sqlalchemy import Column, Integer, String, ForeignKey, Date, Float, Boolean
from sqlalchemy.orm import relationship
from database import Base
```

```
class Country(Base):
    __tablename__ = "countries"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False, unique=True)
    code = Column(String(3), nullable=False, unique=True)
    continent = Column(String(50), nullable=False)

    # Relationships
    leagues = relationship("League", back_populates="country")
    teams = relationship("Team", back_populates="country")
```

```
class League(Base):
    __tablename__ = "leagues"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(150), nullable=False)
    country_id = Column(Integer, ForeignKey("countries.id"), nullable=False)
```

```

level = Column(Integer, nullable=False) # 1 - top division, 2 - second division, etc.
founded_year = Column(Integer)
is_professional = Column(Boolean, default=True)

# Relationships
country = relationship("Country", back_populates="leagues")
teams = relationship("Team", back_populates="league")
matches = relationship("Match", back_populates="league")
seasons = relationship("Season", back_populates="league")

class Team(Base):
    __tablename__ = "teams"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(150), nullable=False)
    city = Column(String(100))
    founded_year = Column(Integer)
    stadium = Column(String(200))
    capacity = Column(Integer)
    country_id = Column(Integer, ForeignKey("countries.id"), nullable=False)
    league_id = Column(Integer, ForeignKey("leagues.id"))

    # Relationships
    country = relationship("Country", back_populates="teams")
    league = relationship("League", back_populates="teams")
    home_matches = relationship("Match", foreign_keys="Match.home_team_id",
back_populates="home_team")
    away_matches = relationship("Match", foreign_keys="Match.away_team_id",
back_populates="away_team")

class Season(Base):
    __tablename__ = "seasons"

    id = Column(Integer, primary_key=True, index=True)
    league_id = Column(Integer, ForeignKey("leagues.id"), nullable=False)
    year_start = Column(Integer, nullable=False)
    year_end = Column(Integer, nullable=False)
    number_of_teams = Column(Integer)
    champion_team_id = Column(Integer, ForeignKey("teams.id"), nullable=True)

    # Relationships
    league = relationship("League", back_populates="seasons")
    champion = relationship("Team", foreign_keys=[champion_team_id])
    matches = relationship("Match", back_populates="season")

```

```

class Match(Base):
    __tablename__ = "matches"

    id = Column(Integer, primary_key=True, index=True)
    season_id = Column(Integer, ForeignKey("seasons.id"), nullable=False)
    league_id = Column(Integer, ForeignKey("leagues.id"), nullable=False)
    home_team_id = Column(Integer, ForeignKey("teams.id"), nullable=False)
    away_team_id = Column(Integer, ForeignKey("teams.id"), nullable=False)
    match_date = Column(Date, nullable=False)
    home_goals = Column(Integer)
    away_goals = Column(Integer)
    attendance = Column(Integer)
    is_played = Column(Boolean, default=True)

    # Relationships
    season = relationship("Season", back_populates="matches")
    league = relationship("League", back_populates="matches")
    home_team = relationship("Team", foreign_keys=[home_team_id])
    away_team = relationship("Team", foreign_keys=[away_team_id])

```

Main.py

```

from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy.orm import Session
from typing import List, Optional
from datetime import date

from database import get_db, engine, Base
from models import Country, League, Team, Season, Match
from crud import *
from pydantic import BaseModel

# Create tables
Base.metadata.create_all(bind=engine)

app = FastAPI(title="European Football Championships API")

# Pydantic models for request/response
class CountryCreate(BaseModel):
    name: str
    code: str
    continent: str

class CountryResponse(BaseModel):
    id: int
    name: str

```

```

code: str
continent: str

class LeagueCreate(BaseModel):
    name: str
    country_id: int
    level: int
    founded_year: Optional[int] = None

class LeagueResponse(BaseModel):
    id: int
    name: str
    country_id: int
    level: int
    founded_year: Optional[int] = None

class TeamCreate(BaseModel):
    name: str
    city: str
    country_id: int
    league_id: Optional[int] = None
    founded_year: Optional[int] = None
    stadium: Optional[str] = None
    capacity: Optional[int] = None

class TeamResponse(BaseModel):
    id: int
    name: str
    city: str
    country_id: int
    league_id: Optional[int] = None
    founded_year: Optional[int] = None
    stadium: Optional[str] = None
    capacity: Optional[int] = None

class SeasonCreate(BaseModel):
    league_id: int
    year_start: int
    year_end: int
    number_of_teams: Optional[int] = None
    champion_team_id: Optional[int] = None

class SeasonResponse(BaseModel):
    id: int
    league_id: int
    year_start: int

```

```

    year_end: int
    number_of_teams: Optional[int] = None
    champion_team_id: Optional[int] = None

class MatchCreate(BaseModel):
    season_id: int
    league_id: int
    home_team_id: int
    away_team_id: int
    match_date: date
    home_goals: Optional[int] = None
    away_goals: Optional[int] = None
    attendance: Optional[int] = None
    is_played: bool = True

class MatchResponse(BaseModel):
    id: int
    season_id: int
    league_id: int
    home_team_id: int
    away_team_id: int
    match_date: date
    home_goals: Optional[int] = None
    away_goals: Optional[int] = None
    attendance: Optional[int] = None
    is_played: bool

# Countries endpoints
@app.post("/countries/", response_model=CountryResponse)
def create_country_endpoint(country: CountryCreate, db: Session = Depends(get_db)):
    return create_country(db, country.name, country.code, country.continent)

@app.get("/countries/", response_model=List[CountryResponse])
def read_countries(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    return get_countries(db, skip=skip, limit=limit)

@app.get("/countries/{country_id}", response_model=CountryResponse)
def read_country(country_id: int, db: Session = Depends(get_db)):
    db_country = get_country(db, country_id)
    if db_country is None:
        raise HTTPException(status_code=404, detail="Country not found")
    return db_country

@app.put("/countries/{country_id}", response_model=CountryResponse)
def update_country_endpoint(country_id: int, name: str = None, code: str = None, continent: str = None, db: Session = Depends(get_db)):

```

```

db_country = update_country(db, country_id, name, code, continent)
if db_country is None:
    raise HTTPException(status_code=404, detail="Country not found")
return db_country

@app.delete("/countries/{country_id}")
def delete_country_endpoint(country_id: int, db: Session = Depends(get_db)):
    success = delete_country(db, country_id)
    if not success:
        raise HTTPException(status_code=404, detail="Country not found")
    return {"message": "Country deleted successfully"}

# Leagues endpoints
@app.post("/leagues/", response_model=LeagueResponse)
def create_league_endpoint(league: LeagueCreate, db: Session = Depends(get_db)):
    return create_league(db, league.name, league.country_id, league.level, league.founded_year)

@app.get("/leagues/", response_model=List[LeagueResponse])
def read_leagues(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    return get_leagues(db, skip=skip, limit=limit)

@app.get("/leagues/{league_id}", response_model=LeagueResponse)
def read_league(league_id: int, db: Session = Depends(get_db)):
    db_league = get_league(db, league_id)
    if db_league is None:
        raise HTTPException(status_code=404, detail="League not found")
    return db_league

@app.put("/leagues/{league_id}", response_model=LeagueResponse)
def update_league_endpoint(league_id: int, name: str = None, level: int = None, founded_year:
int = None, db: Session = Depends(get_db)):
    db_league = update_league(db, league_id, name=name, level=level, founded_year=founded_year)
    if db_league is None:
        raise HTTPException(status_code=404, detail="League not found")
    return db_league

@app.delete("/leagues/{league_id}")
def delete_league_endpoint(league_id: int, db: Session = Depends(get_db)):
    success = delete_league(db, league_id)
    if not success:
        raise HTTPException(status_code=404, detail="League not found")
    return {"message": "League deleted successfully"}

# Teams endpoints
@app.post("/teams/", response_model=TeamResponse)
def create_team_endpoint(team: TeamCreate, db: Session = Depends(get_db)):

```

```

        return create_team(db, team.name, team.city, team.country_id, team.league_id,
                           team.founded_year, team.stadium, team.capacity)

@app.get("/teams/", response_model=List[TeamResponse])
def read_teams(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    return get_teams(db, skip=skip, limit=limit)

@app.get("/teams/{team_id}", response_model=TeamResponse)
def read_team(team_id: int, db: Session = Depends(get_db)):
    db_team = get_team(db, team_id)
    if db_team is None:
        raise HTTPException(status_code=404, detail="Team not found")
    return db_team

@app.put("/teams/{team_id}", response_model=TeamResponse)
def update_team_endpoint(team_id: int, name: str = None, city: str = None,
                        stadium: str = None, capacity: int = None, db: Session =
Depends(get_db)):
    db_team = update_team(db, team_id, name=name, city=city, stadium=stadium, capacity=capacity)
    if db_team is None:
        raise HTTPException(status_code=404, detail="Team not found")
    return db_team

@app.delete("/teams/{team_id}")
def delete_team_endpoint(team_id: int, db: Session = Depends(get_db)):
    success = delete_team(db, team_id)
    if not success:
        raise HTTPException(status_code=404, detail="Team not found")
    return {"message": "Team deleted successfully"}

# Seasons endpoints
@app.post("/seasons/", response_model=SeasonResponse)
def create_season_endpoint(season: SeasonCreate, db: Session = Depends(get_db)):
    return create_season(db, season.league_id, season.year_start, season.year_end,
                        season.number_of_teams, season.champion_team_id)

@app.get("/seasons/", response_model=List[SeasonResponse])
def read_seasons(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    return get_seasons(db, skip=skip, limit=limit)

@app.get("/seasons/{season_id}", response_model=SeasonResponse)
def read_season(season_id: int, db: Session = Depends(get_db)):
    db_season = get_season(db, season_id)
    if db_season is None:
        raise HTTPException(status_code=404, detail="Season not found")
    return db_season

```



```

@app.put("/seasons/{season_id}", response_model=SeasonResponse)
def update_season_endpoint(season_id: int, champion_team_id: int = None,
                           number_of_teams: int = None, db: Session = Depends(get_db)):
    db_season = update_season(db, season_id, champion_team_id=champion_team_id,
                              number_of_teams=number_of_teams)
    if db_season is None:
        raise HTTPException(status_code=404, detail="Season not found")
    return db_season

@app.delete("/seasons/{season_id}")
def delete_season_endpoint(season_id: int, db: Session = Depends(get_db)):
    success = delete_season(db, season_id)
    if not success:
        raise HTTPException(status_code=404, detail="Season not found")
    return {"message": "Season deleted successfully"}

# Matches endpoints
@app.post("/matches/", response_model=MatchResponse)
def create_match_endpoint(match: MatchCreate, db: Session = Depends(get_db)):
    return create_match(db, match.season_id, match.league_id, match.home_team_id,
                        match.away_team_id, match.match_date, match.home_goals,
                        match.away_goals, match.attendance)

@app.get("/matches/", response_model=List[MatchResponse])
def read_matches(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    return get_matches(db, skip=skip, limit=limit)

@app.get("/matches/{match_id}", response_model=MatchResponse)
def read_match(match_id: int, db: Session = Depends(get_db)):
    db_match = get_match(db, match_id)
    if db_match is None:
        raise HTTPException(status_code=404, detail="Match not found")
    return db_match

@app.put("/matches/{match_id}", response_model=MatchResponse)
def update_match_endpoint(match_id: int, home_goals: int = None,
                           away_goals: int = None, attendance: int = None,
                           is_played: bool = None, db: Session = Depends(get_db)):
    db_match = update_match(db, match_id, home_goals=home_goals,
                            away_goals=away_goals, attendance=attendance,
                            is_played=is_played)
    if db_match is None:
        raise HTTPException(status_code=404, detail="Match not found")
    return db_match

```

```

@app.delete("/matches/{match_id}")
def delete_match_endpoint(match_id: int, db: Session = Depends(get_db)):
    success = delete_match(db, match_id)
    if not success:
        raise HTTPException(status_code=404, detail="Match not found")
    return {"message": "Match deleted successfully"}

# Advanced queries
@app.get("/league-table/{season_id}")
def get_league_table_endpoint(season_id: int, db: Session = Depends(get_db)):
    table = get_league_table(db, season_id)
    # Add team names to the table
    for entry in table:
        team = get_team(db, entry['team_id'])
        entry['team_name'] = team.name if team else "Unknown"
    return table

@app.get("/")
def root():
    return {
        "message": "European Football Championships API",
        "docs": "http://127.0.0.1:8000/docs",
        "endpoints": {
            "countries": "/countries/",
            "leagues": "/leagues/",
            "teams": "/teams/",
            "seasons": "/seasons/",
            "matches": "/matches/",
            "league_table": "/league-table/{season_id}"
        }
    }

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="127.0.0.1", port=8000)

```

database.py

```

from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

SQLALCHEMY_DATABASE_URL = "sqlite:///./football_championships.db"

engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False}
)

```

```
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

```
Base = declarative_base()
```

```
def get_db():  
    db = SessionLocal()  
    try:  
        yield db  
    finally:  
        db.close()
```

Crud.py

```
from sqlalchemy.orm import Session  
from sqlalchemy import and_, or_  
from models import Country, League, Team, Season, Match  
from datetime import date  
from typing import Optional, List
```

```
# Country CRUD
```

```
def create_country(db: Session, name: str, code: str, continent: str):  
    country = Country(name=name, code=code, continent=continent)  
    db.add(country)  
    db.commit()  
    db.refresh(country)  
    return country
```

```
def get_countries(db: Session, skip: int = 0, limit: int = 100):  
    return db.query(Country).offset(skip).limit(limit).all()
```

```
def get_country(db: Session, country_id: int):  
    return db.query(Country).filter(Country.id == country_id).first()
```

```
def update_country(db: Session, country_id: int, name: str = None, code: str = None, continent:  
str = None):  
    country = get_country(db, country_id)  
    if country:  
        if name:  
            country.name = name  
        if code:  
            country.code = code  
        if continent:  
            country.continent = continent  
    db.commit()
```

```

        db.refresh(country)
    return country

def delete_country(db: Session, country_id: int):
    country = get_country(db, country_id)
    if country:
        db.delete(country)
        db.commit()
        return True
    return False

# League CRUD
def create_league(db: Session, name: str, country_id: int, level: int, founded_year: int =
None):
    league = League(name=name, country_id=country_id, level=level, founded_year=founded_year)
    db.add(league)
    db.commit()
    db.refresh(league)
    return league

def get_leagues(db: Session, skip: int = 0, limit: int = 100):
    return db.query(League).offset(skip).limit(limit).all()

def get_league(db: Session, league_id: int):
    return db.query(League).filter(League.id == league_id).first()

def update_league(db: Session, league_id: int, **kwargs):
    league = get_league(db, league_id)
    if league:
        for key, value in kwargs.items():
            if hasattr(league, key) and value is not None:
                setattr(league, key, value)
        db.commit()
        db.refresh(league)
    return league

def delete_league(db: Session, league_id: int):
    league = get_league(db, league_id)
    if league:
        db.delete(league)

```

```

        db.commit()
        return True
    return False

# Team CRUD
def create_team(db: Session, name: str, city: str, country_id: int, league_id: int = None,
                founded_year: int = None, stadium: str = None, capacity: int = None):
    team = Team(name=name, city=city, country_id=country_id, league_id=league_id,
                founded_year=founded_year, stadium=stadium, capacity=capacity)
    db.add(team)
    db.commit()
    db.refresh(team)
    return team

def get_teams(db: Session, skip: int = 0, limit: int = 100):
    return db.query(Team).offset(skip).limit(limit).all()

def get_team(db: Session, team_id: int):
    return db.query(Team).filter(Team.id == team_id).first()

def update_team(db: Session, team_id: int, **kwargs):
    team = get_team(db, team_id)
    if team:
        for key, value in kwargs.items():
            if hasattr(team, key) and value is not None:
                setattr(team, key, value)
        db.commit()
        db.refresh(team)
    return team

def delete_team(db: Session, team_id: int):
    team = get_team(db, team_id)
    if team:
        db.delete(team)
        db.commit()
        return True
    return False

# Season CRUD
def create_season(db: Session, league_id: int, year_start: int, year_end: int,

```

```

        number_of_teams: int = None, champion_team_id: int = None):
    season = Season(league_id=league_id, year_start=year_start, year_end=year_end,
                    number_of_teams=number_of_teams, champion_team_id=champion_team_id)
    db.add(season)
    db.commit()
    db.refresh(season)
    return season

def get_seasons(db: Session, skip: int = 0, limit: int = 100):
    return db.query(Season).offset(skip).limit(limit).all()

def get_season(db: Session, season_id: int):
    return db.query(Season).filter(Season.id == season_id).first()

def update_season(db: Session, season_id: int, **kwargs):
    season = get_season(db, season_id)
    if season:
        for key, value in kwargs.items():
            if hasattr(season, key) and value is not None:
                setattr(season, key, value)
        db.commit()
        db.refresh(season)
    return season

def delete_season(db: Session, season_id: int):
    season = get_season(db, season_id)
    if season:
        db.delete(season)
        db.commit()
        return True
    return False

# Match CRUD
def create_match(db: Session, season_id: int, league_id: int, home_team_id: int,
                away_team_id: int, match_date: date, home_goals: int = None,
                away_goals: int = None, attendance: int = None):
    match = Match(season_id=season_id, league_id=league_id, home_team_id=home_team_id,
                  away_team_id=away_team_id, match_date=match_date, home_goals=home_goals,
                  away_goals=away_goals, attendance=attendance)
    db.add(match)
    db.commit()

```

```

db.refresh(match)
return match

def get_matches(db: Session, skip: int = 0, limit: int = 100):
    return db.query(Match).offset(skip).limit(limit).all()

def get_match(db: Session, match_id: int):
    return db.query(Match).filter(Match.id == match_id).first()

def update_match(db: Session, match_id: int, **kwargs):
    match = get_match(db, match_id)
    if match:
        for key, value in kwargs.items():
            if hasattr(match, key) and value is not None:
                setattr(match, key, value)
        db.commit()
        db.refresh(match)
    return match

def delete_match(db: Session, match_id: int):
    match = get_match(db, match_id)
    if match:
        db.delete(match)
        db.commit()
        return True
    return False

# Advanced queries
def get_league_table(db: Session, season_id: int):
    """Get league table for a specific season"""
    matches = db.query(Match).filter(Match.season_id == season_id, Match.is_played ==
True).all()

    table = {}
    for match in matches:
        # Home team
        if match.home_team_id not in table:
            table[match.home_team_id] = {'team_id': match.home_team_id, 'played': 0, 'won': 0,
'drawn': 0, 'lost': 0,
                                     'gf': 0, 'ga': 0, 'points': 0}

```

```

# Away team
if match.away_team_id not in table:
    table[match.away_team_id] = {'team_id': match.away_team_id, 'played': 0, 'won': 0,
'drawn': 0, 'lost': 0,
                                'gf': 0, 'ga': 0, 'points': 0}

# Update home team stats
table[match.home_team_id]['played'] += 1
table[match.home_team_id]['gf'] += match.home_goals
table[match.home_team_id]['ga'] += match.away_goals

# Update away team stats
table[match.away_team_id]['played'] += 1
table[match.away_team_id]['gf'] += match.away_goals
table[match.away_team_id]['ga'] += match.home_goals

if match.home_goals > match.away_goals:
    table[match.home_team_id]['won'] += 1
    table[match.home_team_id]['points'] += 3
    table[match.away_team_id]['lost'] += 1
elif match.home_goals < match.away_goals:
    table[match.away_team_id]['won'] += 1
    table[match.away_team_id]['points'] += 3
    table[match.home_team_id]['lost'] += 1
else:
    table[match.home_team_id]['drawn'] += 1
    table[match.away_team_id]['drawn'] += 1
    table[match.home_team_id]['points'] += 1
    table[match.away_team_id]['points'] += 1

return sorted(table.values(), key=lambda x: (-x['points'], x['gd']))

```


Результат выполнения программы:

POST	/countries/	Create Country Endpoint	▼
GET	/countries/	Read Countries	▼
GET	/countries/{country_id}	Read Country	▼
PUT	/countries/{country_id}	Update Country Endpoint	▼
DELETE	/countries/{country_id}	Delete Country Endpoint	▼
POST	/leagues/	Create League Endpoint	▼
GET	/leagues/	Read Leagues	▼
GET	/leagues/{league_id}	Read League	▼
PUT	/leagues/{league_id}	Update League Endpoint	▼
DELETE	/leagues/{league_id}	Delete League Endpoint	▼
POST	/teams/	Create Team Endpoint	▼
GET	/teams/	Read Teams	▼
GET	/teams/{team_id}	Read Team	▼
PUT	/teams/{team_id}	Update Team Endpoint	▼
DELETE	/teams/{team_id}	Delete Team Endpoint	▼
POST	/seasons/	Create Season Endpoint	▼

```
[{"id":1,"name":"Portugal","code":"POR","continent":"Europe"}, {"id":2,"name":"Spain","code":"SPA","continent":"Europe"}, {"id":3,"name":"BELARUS","code":"BEL","continent":"Europe"}]
```

```
[{"id":1,"name":"FC Barcelona","city":"Barcelona","country_id":2,"league_id":2,"founded_year":1899,"stadium":"Camp Nou","capacity":99354},
```

```
[{"id":1,"name":"Primeira Liga","country_id":6,"level":1,"founded_year":1934}, {"id":2,"name":"Primeira Liga","country_id":6,"level":1,"founded_year":1934}]
```

Вывод: приобрел практические навыки разработки API и баз данных.